

QualiJournal Admin Handover Report (업무 인수인계)

1. 시스템 개요

QualiJournal 서비스는 하루에 하나의 키워드를 정해 다양한 출처(뉴스, 블로그, 학술자료, 커뮤니티 등)의 콘텐츠를 수집하고, 편집자의 승인 과정을 거쳐 특집 기사로 발행하는 플랫폼입니다. 시스템은 두 축으로 구성됩니다:

- **오케스트레이션 파이프라인** - `orchestrator.py` 를 중심으로 키워드별 뉴스 수집, 분석, 요약 및 발행을 담당합니다. `collect_news.py` 등 모듈이 데이터를 수집하여 최종 Markdown/HTML 페이지를 생성하고 아카이브에 저장합니다.
- **관리자(Admin) 시스템** - 백엔드 `server_quali.py` 와 단일 HTML/JS 프론트엔드(`admin/index.html`)로 이루어진 관리자 UI입니다. 편집자는 이 UI를 통해 기사 승인과 발행 작업을 수행하고, 일일 보고서 생성이나 자료 번역/요약 등의 API를 호출할 수 있습니다.

2. 배포 및 인프라 환경

2.1 GCP Cloud Run 서비스

- **프로젝트 ID:** `quali-journal-prod`
- **서비스:**
 - `quali-admin-domap` - 커스텀 도메인(`admin.standardai.co.kr`)에 매핑된 프론트-API 서비스로, 리전은 도쿄(`asia-northeast1`).
 - `quali-journal-admin` - 백오피스(소스 빌드) 서비스로, 리전은 서울(`asia-northeast3`).
- **도메인 매핑:** `admin.standardai.co.kr` → `quali-admin-domap`. 이 매핑을 `gcloud beta run domain-mappings describe` 명령으로 확인할 수 있습니다. 서비스 변경 후 최신 리비전으로 트래픽을 100 % 전환하는 것이 런북의 기본 절차입니다.

2.2 CI/CD

- **GitHub Actions + OIDC:** 메인 브랜치에 커밋이 푸시되면 GitHub Actions가 트리거되어 도커 이미지 빌드 → Artifact Registry 푸시 → Cloud Run 배포를 수행합니다.
- **비밀 주입:** `ADMIN_TOKEN` 및 `API_KEY` 와 같은 민감한 환경변수는 Secret Manager를 통해 주입합니다. `.env` 파일은 개발/배포 환경에서 로드되며 Git에 커밋되지 않습니다.
- **배포 후 스모크 테스트:** Workflows는 배포 후 `/health` 와 `/api/status` 등의 엔드포인트를 호출해 200/401 응답을 확인합니다.

2.3 배포 검증 및 트래픽 전환

런북에서는 새 리비전 배포 후 `gcloud run services describe`로 리비전 상태를 조회하고, 필요하면 `gcloud run services update-traffic` 명령으로 최신 리비전에 트래픽을 100 % 할당하도록 안내합니다. 배포 후 캐시로 인해 UI가 갱신되지 않는 문제를 방지하기 위해 브라우저를 `?v=<random>` 쿼리로 새로고침하거나 캐시를 무시하고 로드해야 합니다.

3. 코드베이스 및 주요 구성 파일

3.1 서버 (FastAPI)

- **server_quali.py** - 관리자 API 서버 엔트리포인트. `/health` (헬스체크), `/api/status` (관리자 토큰 검증용), `/api/report` (일일 보고서 생성/조회), `/api/enrich` (요약·번역), `/api/export` (보고서 다운로드) 등의 라우트가 정의되어 있습니다.
- **orchestrator.py** - 뉴스 수집·발행 파이프라인 전체를 실행하는 스크립트. 키워드 수집과 평가, 요약, 발행을 수행합니다.
- **make_daily_report.py** - `/api/report` 에서 사용되는 일일 보고서 생성 로직을 포함합니다.

3.2 프론트엔드

`admin/index.html` 과 관련 JS는 관리자 UI를 구현합니다. 프론트단에서는 `ADMIN_TOKEN` 입력 창을 제공하고, 요청을 보낼 때 `Authorization: Bearer <TOKEN>` 또는 `X-Admin-Token` 헤더를 자동으로 첨부하여 API를 호출합니다.

3.3 Dockerfile 및 ignore 설정

- **Dockerfile** - Python 3.11 slim 베이스 이미지로 전체 소스를 `/app` 아래에 복사하고 `uvicorn` + `gunicorn` 으로 `server_quali:app` 을 실행합니다. 패치 시 루트 `admin/index.html` 을 `/app/index.html` 로 복사하고 낡은 `/app/dist` 폴더를 삭제하여 이전 빌드 산출물이 UI를 덮어쓰지 않도록 했습니다.
- **.dockerignore / .gcloudignore** - HTML/JS/CSS 및 `admin/**`, `server_quali.py`, `orchestrator.py` 등 필요한 파일을 포함시킨 예외 규칙을 작성하여 빌드 이미지에 정적 자산이 빠지지 않도록 구성했습니다. `dist/` 폴더는 항상 제외해 스티커 버전을 배포하지 않도록 했습니다.

4. 수정 및 개선 내역

4.1 Docker 이미지 경로 문제 해결

초기 배포에서 UI가 업데이트되지 않는 문제가 발생했습니다. 이는 도커 컨테이너에 이전 빌드의 `/app/dist/index.html` 이 남아 있어 새 `index.html` 이 서빙되지 않은 것이 원인이었습니다. 해결을 위해 Dockerfile에 아래 RUN 스크립트를 추가했습니다:

```
# 최신 UI 보장: admin/index.html을 최종본으로 강제 + 낡은 dist 제거
RUN set -eux; \
  if [ -f "/app/admin/index.html" ]; then \
    cp -f /app/admin/index.html /app/index.html; \
  fi; \
  rm -rf /app/dist
```

이 명령은 이미지 빌드 시 항상 최신 `admin/index.html` 을 `/app/index.html` 로 복사하고, 과거 빌드 산출물인 `dist/` 를 삭제해 최신 UI가 서빙되도록 보장합니다. `.dockerignore` 와 `.gcloudignore` 도 HTML/JS/CSS 및 `admin/**` 를 포함시키도록 업데이트했습니다.

4.2 환경변수 및 Secret 관리

- `ADMIN_TOKEN` 과 `API_KEY` 는 Secret Manager에서 주입하며, `.env` 파일에는 실제 값 대신 예시만 저장합니다. 배포 시 `--set-secrets "ADMIN_TOKEN=ADMIN_TOKEN:latest"` 옵션을 사용해 주입합니다.
- Cloud Run 환경에서 쓰기 가능한 경로는 `/tmp` 뿐입니다. 문서에서도 Cloud Run에서는 `/tmp/archive` 경로를 사용하도록 ENV를 설정해야 한다고 강조합니다.
- 따라서 백엔드 코드에서 `ARCHIVE_DIR` 환경변수를 읽어 `/tmp/archive` 를 사용하도록 수정하였습니다. 또한 `REPORT_DIR` 을 `/tmp/archive/reports` 로 설정하여 일일 보고서 저장 경로를 지정했습니다.

4.3 server_quali.py 수정

- **아카이브 경로 선택 함수 추가** - Cloud Run인지 여부를 `K_SERVICE` 환경변수로 판단하여, `ARCHIVE` 대신 쓰기 가능한 `/tmp/archive` 를 선택하는 `_archive_base()` 함수를 추가했습니다.
- **날짜 해석 헬퍼** `_resolve_day()` - GET 과 POST 의 반복 코드를 통일하고, 전달되지 않은 경우 오늘 날짜를 ISO 형식으로 반환하도록 구현했습니다.
- **GET /api/report** - 기존 코드는 전역 `ARCHIVE` 디렉터리에서 파일을 찾았는데, Cloud Run에서는 `/tmp/archive` 생성 권한이 없어 500 에러가 발생했습니다. 수정 후 코드에서 `base = _archive_base()` 를 사용하고 `base.glob()` 으로 파일 목록을 조회합니다. 결과를 중복 없이 수집하고 이름순으로 정렬하여 반환합니다.
- **POST /api/report** - 날짜 해석을 `_resolve_day()` 로 통일하고, 보고서 생성 후 `/tmp/archive/reports` 에 Markdown 파일을 저장하도록 개선했습니다.

이러한 수정으로 Cloud Run 환경에서도 `/api/report` 호출 시 200을 반환하며 보고서가 정상 생성됩니다. 문서는 관리자가 UI에서 “보고서 생성” 버튼을 누르거나 `/api/report` 엔드포인트를 호출해 일일 보고서를 얻을 수 있다고 안내합니다.

4.4 배포/테스트 절차 개선

운영 가이드에 따라 다음과 같은 절차를 따라 배포와 검증을 수행했습니다:

1. **도메인 매핑 확인** - `gcloud beta run domain-mappings describe --domain admin.standardai.co.kr --region asia-northeast1` 로 `routeName` 이 `quali-admin-domap` 인지 확인.
2. **환경변수 확인** - `gcloud run services describe <service> --region <region> --format="value(spec.template.spec.containers[0].env[].name)"` 명령으로 `ADMIN_TOKEN`, `ARCHIVE_DIR` 등 필요한 키가 주입되었는지 검증.
3. **헬스 체크** - 배포 후 `curl -s -o NUL -w "%{http_code}\n" "$URL/health"` 가 200인지 확인. `/api/status` 와 `/api/report` 는 `Authorization: Bearer <ADMIN_TOKEN>` 헤더를 붙여 200이 나오는지 확인하며, 토큰 없이 호출하면 401이 나와야 합니다.
4. **최신 리비전 트래픽 전환** - `gcloud run services update-traffic quali-admin-domap --to-latest --region asia-northeast1` 로 트래픽을 최신 리비전에 할당하고, `gcloud run services describe` 로 `latestRevision: true` 와 `percent: 100` 을 확인.
5. **캐시 무효화** - 새 UI가 반영되지 않을 경우 `https://admin.standardai.co.kr/?v=<무작위숫자>` 로 접속하거나 개발자 도구에서 캐시를 비활성화하여 새로고침해야 합니다.

5. 트러블슈팅 및 팁

- **/api/report 500 에러** - 로그에서 `PermissionError: [Errno 13] Permission denied: '/archive'` 가 발생한 경우 Cloud Run에서는 `/app` 경로가 읽기 전용임을 의미합니다. `ARCHIVE_DIR=/tmp/archive` 환경변수를 설정하고 `_archive_base()` 를 사용해 `/tmp/archive` 를 디렉터리 생성 및 파일 조희 경로로 지정하면 해결됩니다.
- **UI 버그 패치 미적용** - CI가 성공했지만 라이브 서비스에 새 빌드가 배포되지 않아 UI 변경사항이 반영되지 않는 경우, 최신 리비전으로 트래픽을 전환하고 브라우저 캐시를 무효화해야 합니다. 해당 흐름은 2025-10-22 문제 해결 런북에 정리되어 있습니다.
- **Secrets 관리** - `.env` 파일은 Git에 커밋하지 않으며, 예시 파일만 버전에 포함합니다. Secret Manager를 통해 주입되는 `ADMIN_TOKEN`, `API_KEY` 등의 값은 로컬에서 유출되지 않도록 관리합니다.

6. 향후 개선 제안

1. **CI 파이프라인에 테스트 추가** - 현재는 배포 스크립트가 기본적인 헬스 체크만 수행합니다. `/api/report` 등 주요 API에 대한 통합 테스트를 워크플로에 추가하여 기능 이상 여부를 자동으로 검증하면 안정성이 향상됩니다.
2. **모니터링 및 알림** - Cloud Run 서비스에 대한 헬스 모니터링을 Cloud Monitoring과 연동하여 응답 코드 이상이나 트래픽 불일치가 감지될 때 즉시 알림을 받을 수 있도록 설정합니다.
3. **문서 업데이트 자동화** - `TAGGING.md` 규칙에 따라 각 파일 상단에 컴포넌트와 기능 앵커를 주석으로 기록하면, SSOT 및 인벤토리를 자동 갱신할 수 있습니다.
4. **UI/UX 개선** - 관리자 UI에서 인증 토큰을 세션 스토리지에 안전하게 저장하고 토큰 유효기간 만료 시 자동 갱신 기능을 구현하면 사용자 편의성이 높아질 것입니다.

7. 마무리

본 보고서는 현재 채팅에서 수행한 모든 작업과 수정사항을 기반으로 작성되었습니다. 주요 개선사항은 Docker 이미지의 UI 패치 반영, 서버 코드의 Cloud Run 호환성 강화(`/tmp/archive` 사용), 일일 보고서 API 안정화, 환경변수/비밀 관리 체계 정비 등입니다. 새로운 채팅에서도 이 보고서를 참고하여 작업을 이어가면 동일한 문제를 반복하지 않고 지속적으로 시스템을 개선할 수 있을 것입니다.
